

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-209БВ-25

Студент: Девлетбиев С.Р.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 23.09.26

Москва, 2026

Постановка задачи

Вариант 12, группа 3. Написать программу на C++ для Linux. Родительский процесс принимает строки произвольной длины и передаёт их двум дочерним процессам. Первый переводит латинские буквы в верхний регистр, второй убирает повторные пробелы. Результат возвращается родителю и выводится на экран.

Передача данных между процессами выполняется через неименованные каналы. Для ввода и вывода используются системные вызовы, без `stdio`. Строки хранятся в `std::string`.

Общий метод и алгоритм решения

Используемые системные вызовы

`pipe(fd)` создаёт канал. Из `fd[0]` читают данные, в `fd[1]` записывают.

`fork()` создаёт дочерний процесс. Возвращаемое значение позволяет определить, какой процесс выполняет код.

`dup2(oldfd, newfd)` перенаправляет стандартный ввод или вывод в канал.

`execl(path, arg0, ...)` запускает программу `child1` или `child2` в дочернем процессе.

`read(fd, buffer, count)` читает данные, `write(fd, buffer, count)` записывает их. При неполной записи `writeAll` вызывает `write` ещё раз.

`close(fd)` закрывает ненужный дескриптор канала.

`waitpid(pid, &status, 0)` ждёт завершения дочернего процесса.

`_exit(code)` завершает дочерний процесс, если не удалось настроить каналы или выполнить `execl`.

Алгоритм

1. Родитель создаёт три канала: `pipe1` передаёт данные в `child1`, `pipeMiddle` соединяет `child1` с `child2`, `pipe2` возвращает результат родителю.
2. Через `fork` создаются два дочерних процесса. Каждый подключает каналы к `stdin` и `stdout` с помощью `dup2`, закрывает лишние дескрипторы и запускает свою программу через `execl`.
3. Родитель читает строку через `readLine`, записывает её в `pipe1` через `writeAll`, читает ответ из `pipe2` и выводит его.
4. `child1` переводит латинские буквы от `a` до `z` в верхний регистр. `child2` заменяет последовательность обычных пробелов одним пробелом.
5. После окончания ввода родитель закрывает каналы и дожидается завершения обоих дочерних процессов.

`readLine` читает строку до перевода строки или EOF, поэтому её длина не ограничена размером буфера. Регистр меняется только у латинских букв ASCII. Кириллица и табуляция остаются без изменений. Повторным пробелом считается символ ASCII 0x20.

Код программы

Все файлы находятся в одном каталоге. Перед командами в Makefile стоят символы табуляции.

parent.cpp

```
#include "io_utils.h"
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <string>

int main() {
    int pipe1[2], pipeMiddle[2], pipe2[2];
    if (pipe(pipe1) < 0) return 1;
    if (pipe(pipeMiddle) < 0) {
        close(pipe1[0]); close(pipe1[1]);
        return 1;
    }
    if (pipe(pipe2) < 0) {
        close(pipe1[0]); close(pipe1[1]);
        close(pipeMiddle[0]); close(pipeMiddle[1]);
        return 1;
    }

    pid_t pid1 = fork();
    if (pid1 < 0) return 1;
    if (pid1 == 0) {
        if (dup2(pipe1[0], STDIN_FILENO) < 0 ||
            dup2(pipeMiddle[1], STDOUT_FILENO) < 0) _exit(1);
        close(pipe1[0]); close(pipe1[1]);
        close(pipeMiddle[0]); close(pipeMiddle[1]);
        close(pipe2[0]); close(pipe2[1]);
        execl("./child1", "child1", (char*)nullptr);
        _exit(1);
    }

    pid_t pid2 = fork();
    if (pid2 < 0) {
        close(pipe1[0]); close(pipe1[1]);
        close(pipeMiddle[0]); close(pipeMiddle[1]);
        close(pipe2[0]); close(pipe2[1]);
        int status;
        waitpid(pid1, &status, 0);
        return 1;
    }
    if (pid2 == 0) {
        if (dup2(pipeMiddle[0], STDIN_FILENO) < 0 ||
            dup2(pipe2[1], STDOUT_FILENO) < 0) _exit(1);
        close(pipe1[0]); close(pipe1[1]);
        close(pipeMiddle[0]); close(pipeMiddle[1]);
        close(pipe2[0]); close(pipe2[1]);
        execl("./child2", "child2", (char*)nullptr);
        _exit(1);
    }

    close(pipe1[0]);
    close(pipeMiddle[0]); close(pipeMiddle[1]);
    close(pipe2[1]);

    std::string line, result;
    int input_status;
    bool ok = true;
    bool input_closed = false;
    while ((input_status = readLine(STDIN_FILENO, line)) > 0) {
        bool final_line = !line.empty() && line.back() != '\n';
        if (writeAll(pipe1[1], line.data(), line.size()) < 0) {
            ok = false;
            break;
        }
        if (final_line) {
            close(pipe1[1]);
            input_closed = true;
        }
        if (
            readLine(pipe2[0], result) != 1 ||
            writeAll(STDOUT_FILENO, result.data(), result.size()) < 0) {
            ok = false;
            break;
        }
    }
    if (input_status < 0) ok = false;
    if (!input_closed) close(pipe1[1]);
    close(pipe2[0]);
}
```

```

    int status1 = 0, status2 = 0;
    if (waitpid(pid1, &status1, 0) < 0 ||
        waitpid(pid2, &status2, 0) < 0) ok = false;
    return ok && WIFEXITED(status1) && WEXITSTATUS(status1) == 0 &&
        WIFEXITED(status2) && WEXITSTATUS(status2) == 0 ? 0 : 1;
}

```

child1.cpp

```

#include "io_utils.h"
#include <unistd.h>
#include <string>
#include <cstdint>

int main() {
    std::string line;
    int status;
    while ((status = readLine(STDIN_FILENO, line)) > 0) {
        for (std::size_t i = 0; i < line.size(); ++i) {
            if (line[i] >= 'a' && line[i] <= 'z')
                line[i] = static_cast<char>(line[i] - 'a' + 'A');
        }
        if (writeAll(STDOUT_FILENO, line.data(), line.size()) < 0)
            return 1;
    }
    return status < 0 ? 1 : 0;
}

```

child2.cpp

```

#include "io_utils.h"
#include <unistd.h>
#include <string>
#include <cstdint>

int main() {
    std::string line;
    int status;
    while ((status = readLine(STDIN_FILENO, line)) > 0) {
        std::string result;
        bool previous_space = false;
        for (std::size_t i = 0; i < line.size(); ++i) {
            char c = line[i];
            if (c == ' ') {
                if (previous_space) continue;
                previous_space = true;
            } else {
                previous_space = false;
            }
            result += c;
        }
        if (writeAll(STDOUT_FILENO, result.data(), result.size()) < 0)
            return 1;
    }
    return status < 0 ? 1 : 0;
}

```

io_utils.h

```

#ifndef IO_UTILS_H
#define IO_UTILS_H

#include <string>
#include <cstdint>

int writeAll(int fd, const char* data, std::size_t size);
// 1: строка; 0: EOF; -1: ошибка.
int readLine(int fd, std::string& line);

#endif

```

io_utils.cpp

```

#include "io_utils.h"
#include <unistd.h>
#include <cerrno>

int writeAll(int fd, const char* data, std::size_t size) {
    std::size_t pos = 0;
    while (pos < size) {
        ssize_t n = write(fd, data + pos, size - pos);
        if (n < 0 && errno == EINTR) continue;
        if (n <= 0) return -1;
        pos += static_cast<std::size_t>(n);
    }
    return 0;
}

```

```
}

int readLine(int fd, std::string& line) {
    line.clear();
    char c;
    while (true) {
        ssize_t n = read(fd, &c, 1);
        if (n < 0 && errno == EINTR) continue;
        if (n < 0) return -1;
        if (n == 0) return line.empty() ? 0 : 1;
        line += c;
        if (c == '\n') return 1;
    }
}
```

Makefile

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -Wextra -Wpedantic

all: parent child1 child2

parent: parent.cpp io_utils.cpp io_utils.h
    $(CXX) $(CXXFLAGS) parent.cpp io_utils.cpp -o parent

child1: child1.cpp io_utils.cpp io_utils.h
    $(CXX) $(CXXFLAGS) child1.cpp io_utils.cpp -o child1

child2: child2.cpp io_utils.cpp io_utils.h
    $(CXX) $(CXXFLAGS) child2.cpp io_utils.cpp -o child2

run: all
    ./parent

clean:
    rm -f parent child1 child2

.PHONY: all run clean
```

Протокол работы программы

Примеры запуска из каталога с исходными файлами. Для подачи входных строк используется команда `printf` оболочки.

Тест 1. Регистр и повторные пробелы

```
$ make
$ printf 'Hello    my    friend\n' | ./parent
HELLO MY FRIEND
```

Тест 2. Несколько строк

```
$ printf 'abc  def\nOne two\n' | ./parent
ABC DEF
ONE TWO
```

Тест 3. Пустая строка и ввод без перевода строки

```
$ printf '\nabc  def' | ./parent

ABC DEF
```

Тест 4. Длинная строка

```
$ printf '%0300d xyz\n' 0 | ./parent | wc -c
305
```

В последнем тесте на вход подаются 300 цифр, два пробела, хуз и перевод строки. После удаления лишнего пробела длина вывода составляет 305 байт.

Вывод

Программа обрабатывает строки в двух дочерних процессах и передаёт результат обратно родителю. Проверены строки с несколькими пробелами, несколько строк подряд и ввод без перевода строки в конце. Функции `readLine` и `writeAll` вынесены в отдельные файлы, чтобы не повторять их в каждой программе. При сборке возникла ошибка с отсутствующей целью `make run`; после добавления цели в `Makefile` запуск стал выполняться одной командой.